The Relay: Final Engineering and Experimental Report

1. What I Built and Why

The Relay is an experimental prototype of a fault-tolerant Retrieval-Augmented Generation (RAG) system for academic literature question answering. In networked environments, LLMs face rate limits, latency variance, 5xx errors, and hallucination when context is incomplete.

I built:

1. Lexical Retrieval Engine: In-memory Okapi BM25 index over research papers (`attention_v7.pdf`, `dynamic_chunking_v2.pdf`, `moshi_v1.pdf`).
2. Two-Stage Verification Pipeline (`ResearchPipeline`): Queries an evidence verification gate prior to synthesis, requires bracketed source IDs (`[S1]`, `[S2]`), performs structural citation validation (confirming cited IDs exist in the retrieved set without asserting semantic claim truth), and issues structured refusals when evidence is insufficient.
3. Traffic Control Plane (`RequestController`): Combines a sliding-window `RateLimiter`, a bounded-semaphore `ConcurrencyLimiter`, and exponential backoff `retry`.
4. Deterministic Stand-In Model (`StandInModel`): Injects simulated latencies, bursts, synthetic 429 quota exceptions, and upstream faults without API costs.

2. The Real Service and My Stand-In

Live baseline testing targeted Groq Cloud API's `openai/gpt-oss-20b` via `GroqClient`. Because high-volume load testing against live endpoints exhausts free-tier quotas, I developed `StandInModel` implementing the identical `ModelProvider` protocol (`generate`, `check_evidence`).

- Configurable Latency: Calibrated to nominal baseline response times (configured default: 573 ms).
- Synthetic Fault Injection: Deterministic failure rates, simulated quota exceptions (`rate_limit_every`), refusals, and token streaming.
- Controlled Daily Quota: Wrapped with `DailyBudgetStandIn` to simulate hard daily exhaustion.
- Critical Caveat: Stand-in runs represent client-side simulation. They demonstrate local queuing, concurrency limits, and retry behavior, but do not reproduce actual upstream Groq HTTP 429 response headers, retry-after directives, or remote scheduling queues.

3. Results

Table 1 — Real Groq Limits (Provisional Project Dashboard Values)

Provisional project-dashboard values; not independently verified via remote service saturation during this test run.

Metric	Recorded Value	Status / Source
Requests Per Minute (RPM)	30	Previously recorded project-dashboard limit
Requests Per Day (RPD)	1,000	Previously recorded project-dashboard limit
Tokens Per Minute (TPM)	8,000	Previously recorded project-dashboard throughput limit
Tokens Per Day (TPD)	200,000	Previously recorded project-dashboard throughput limit
Concurrent In-Flight Limit	Not measured	Undocumented / Not measured
Actual Remote Limit Response	Not measured	Live 429 payload structure not saturated
Recovery Time Window	Not measured	Remote quota replenishment window not measured

Table 2a — Latency Across Experimental Scenarios

Calculated directly from `results/results.csv` using `scripts/analyze_load_test.py`.

Scenario	Source	Count	First-Token p50/p95/p99*	Total p50 (ms)	Total p95 (ms)	Total p99 (ms)
`baseline_sequential`	Real Groq	5	Not measured	572.59	2086.69	2363.55
`hour9_steady_v2`	Stand-in	60	Not measured	573.74	575.49	575.89
`hour9_burst`	Stand-in	120	Not measured	3154.31	5734.02	5734.80
`hour9_sustained_overload`	Stand-in	160	Not measured	3104.92	5636.44	6057.19
`hour9_reduced_limit_v1`	Stand-in	40	Not measured	2292.52	61140.12	61153.48
`hour9_daily_budget_v1`	Stand-in	20	Not measured	2647.66	4900.15	4900.53

Table 2b — Reliability, Retries, Queueing, and Costs

Scenario	Success (%)	Rate-Limited (%)	Timeout/Error (%)	Avg Retries	Peak Queue (Observed)*	Avg Tokens In/Out	Cost / 1k Answers**
`baseline_sequential`	100.0%	0.0%	0.0% / 0.0%	0.00	Not measured	189.0 / 264.8	$0.00 (Free Tier)
`hour9_steady_v2`	100.0%	0.0%	0.0% / 0.0%	0.00	~1	24.0 / 18.0	$0.00 (Stand-in)
`hour9_burst`	100.0%	0.0%	0.0% / 0.0%	0.00	~27	24.0 / 18.0	$0.00 (Stand-in)
`hour9_sustained_overload`	100.0%	0.0%	0.0% / 0.0%	0.00	~28	24.0 / 18.0	$0.00 (Stand-in)
`hour9_reduced_limit_v1`	100.0%	0.0%	0.0% / 0.0%	0.00	~17	24.0 / 18.0	$0.00 (Stand-in)
`hour9_daily_budget_v1`	25.0%	75.0%	0.0% / 0.0%	1.50	Not measured	24.0 / 18.0	$0.00 (Stand-in)

\Caveats*: Only 5 real Groq requests measured. First-token latency not measured (non-streaming). Queue peaks marked `~` were approximate console observations, not logged in CSV. Pooled runs prevent deriving single-run arrival rates. `answer_grade` is blank.

\\Cost Distinctions: Actual Groq free-tier testing cost: $0.00. Stand-in API cost: $0.00. Theoretical list-price estimate: ~$0.094 per 1,000 baseline answers (~$0.10/1M prompt, ~$0.30/1M completion tokens; not a verified current price).

4. Where the Time Goes

1. Remote Inference: Total sequential duration on Groq ranged from 298.23 ms to 2432.77 ms (initial cold run). Without separate TTFT instrumentation, total elapsed time reflects network transit plus full response completion.
2. Local Concurrency Queuing: When bursts exceed `max_concurrent=3`, requests wait in the executor pool. In `hour9_burst`, this local queuing expanded p95 total latency to 5734.02 ms while simulated model latency remained 573 ms.
3. Sliding-Window Delays: In the reduced-limit simulation, the local controller enforced a 15-request rolling window over 60 seconds. Requests exceeding available local capacity waited for the window to refresh, producing p95 latency of 61,140.12 ms.
4. Retrieval Overhead: BM25 searches in-memory over 315 tokenized chunks. Isolated retrieval execution timing was not separately benchmarked in the experiment suite.

5. When a Limit Is Hit

- Proactive Local Limiting (`RateLimiter`): When the sliding window is full, `RateLimiter.acquire()` sleeps until capacity clears. This local delay preserves 100% completion in `hour9_reduced_limit_v1` at the cost of high latency, without generating remote 429 errors.
- Reactive Backoff (`RetryPolicy`): Catches retryable errors (including simulated `StandInRateLimitError` and HTTP 429) and executes exponential backoff.
- Hard Quota Exhaustion (`hour9_daily_budget_v1`): When the daily budget model raised quota errors after 5 calls, requests retried up to `max_retries=2` and failed (75% rate-limited, avg 1.5 retries), verifying clean termination when capacity is unrecoverable.

6. Retrieval: Quality Versus Cost

Evaluated across 120 ingested paper pages against 15 benchmark questions (`evaluation/questions.json`):

Setting	Chunk Size	Overlap	Top-K	Chunks	All Expected Papers Found	Term Coverage
Setting A	250 words	50 words	3	315	11 / 13 (84.6%)	92.31%
Setting B	500 words	50 words	5	184	11 / 13 (84.6%)	98.08%
Setting C	800 words	50 words	5	122	12 / 13 (92.3%)	100.00%

Setting C achieved highest paper recall (12/13) and 100% term coverage by passing large passages (up to 4,000 words context). Setting B represents a potential operational compromise based on retrieval coverage alone, achieving 98.08% expected-term coverage and 84.6% expected-paper coverage. Its actual impact on token throughput, generation latency and cost was not benchmarked. The previously recorded 8,000 TPM quota is an aggregate throughput limit across requests, not a per-request context limit.

7. How This Library Will Mislead a Naive Retriever

A naive retriever assumes high BM25 lexical similarity guarantees a passage can answer the query. Diagnostic scoring (`scripts/inspect_scores.py`) demonstrates this failure:

- "What is the best Java programming course?" returned a Moshi chunk (page 63) scoring 13.0191 due to general programming terms.
- "What is the current NVIDIA stock price?" matched the Attention paper (page 15) scoring 10.2110 due to GPU training hardware mentions.

Diagnostic vs Pipeline Distinction: `scripts/inspect_scores.py` isolated BM25 scoring. In full pipeline integration testing (`scripts/test_pipeline.py`), the NVIDIA query was tested end-to-end against real Groq; the evidence gate returned `sufficient: False`, issuing an end-to-end refusal. In contrast, the Java query was evaluated solely as a retrieval-score diagnostic and was not run through the end-to-end pipeline test.

8. Retries

- Targeted Errors: Retries only trigger on transient errors (`StandInError`, `StandInRateLimitError`, `RateLimitError`, `APITimeoutError`, `APIConnectionError`, `TimeoutError`, `ConnectionError`, and status codes 429, 500, 502, 503, 504). Deterministic exceptions (`ValueError`) fail immediately.
- Backoff Formula: $\text{delay} = \min(\text{base_delay} \times 2^{\text{attempt}}, \text{max_delay})$ with defaults `base_delay = 0.25s`, `max_delay = 2.0s`, `max_retries = 2`.
- Verification: In `scripts/test_pipeline_retries.py`, an intentionally flaky provider injected single faults into evidence checking and generation; both stages recovered (exactly 1 retry each) to complete the answer.

9. What the Person Sees

`ResearchPipeline.run()` returns a structured dictionary:

1. Answered (`status: "answered"`): Bracketed citation markers (`[S1]`, `[S2]`), metadata list (`citations`), `retry_metrics`, and `generation_metrics` (`latency_ms`, `tokens_in`, `tokens_out`, `request_id`). Citation validation confirms structural presence in retrieved passages; it does not perform automated NLI claim entailment.
2. Refused (`status: "refused"`): Fired when retrieval yields no passages or `check_evidence` reports insufficient support ("I cannot answer this reliably using the available research papers."). Includes reason, retrieved count, and retry metrics; omits generation metrics.
3. Error (`status: "error"`): Fired on persistent upstream failures ("I could not complete a reliable answer. Please try again."). Includes error reason and retry metrics.

10. What I'm Unsure About, What I Left Out, What I Learned

- Left Out: Dense vector embeddings (avoided external vector databases and embedding latency at the expense of recall on vocabulary mismatches); `asyncio` (used threads for simpler locking); web frontend (focused on reproducible CLI benchmarks).
- Unsure About: True upstream Groq concurrency ceilings for developer tiers remain undocumented; live 429 window replenishment dynamics were not verified via remote saturation.
- Key Learnings: Concurrency limiting must precede rate limiting to avoid thread starvation. High BM25 scores must never be treated as confidence scores for factual answering.

11. Feedback on Vaani

Note: Based solely on developer documentation and architectural specifications. Live audio streams, Hindi voice synthesis, telephony latency, and call handoffs were not tested.

1. Status Hooks: Vaani's separation of telephony from reasoning works well with structured pipeline statuses (`answered`, `refused`, `error`), enabling clean conversational turn-offs rather than raw error trace readouts.
2. Latency Sensitivity: Voice agents require < 800 ms latency. Client-side queuing (3-5s in bursts, 60s in rate limits) creates dead air on calls. Voice pipelines need immediate fallback or fast refusal rather than queue waits.
3. Telemetry: Real-time provider health and queue depth indicators in developer dashboards would allow upstream routing before calls stall.

12. How I Worked With the Assistant

1. Division of Responsibility: Architecture, concurrency rules, rate limiter specs, and evaluation criteria were directed by me. The assistant generated boilerplate code, drafted tests, and formatted tables.
2. Pivotal Interaction Summaries:
3. Prompt Summary 1 (Hour 7): Requesting design of a thread-safe `RequestController` combining a sliding-window rate limiter with a bounded semaphore.
4. Prompt Summary 2 (Hour 9): Designing `StandInModel` to simulate latency, rate limits, and failure modes offline.
5. Prompt Summary 3 (Hour 10): Directing a strict data audit to extract metrics directly from raw CSV outputs without fabricating values.
6. Correcting Assistant Calculations: In an earlier analysis turn, the assistant attempted to calculate offered rates by taking the timestamp span across the full CSV. Because the dataset pooled repeated runs conducted at different times, this produced an erroneous duration. I corrected this logic, instructing the assistant to report counts and latencies without calculating single-run rates from pooled timestamps.
7. Scope Boundaries and Disagreements: The project stayed focused on backend resilience and reproducible CLI benchmarks, avoiding unnecessary frontends. No clearly documented example of an explicit assistant recommendation being overruled was retained in the recorded logs.